FE 515 Lecture 4 Walkthrough

Monte Carlo Simulation in R

Sean O'Leary

Contents

1 Overview **1**

2 Monte Carlo Intuition: Coin Flips **2**

2.1 Simulating Coin Flips in R 2

2.2 Tracking the Estimated Probability 3

3 Monte Carlo for Financial Returns **4**

4 Simulating Stock Returns **4**

4.1 Visualizing the Simulated Distribution 5

5 Estimating a Probability with Monte Carlo **6**

5.1 Monte Carlo Estimate 6

6 Comparing Monte Carlo to the Analytical Answer **7**

7 Simulating Portfolio Profit and Loss **8**

8 A Simple Value at Risk Calculation **9**

9 Analytical VaR with `qnorm()` **10**

10 Summary **11**

1 Overview

In this walkthrough, we will use simulation to answer probabilistic and financial questions.

The main idea behind Monte Carlo simulation is simple:

$$\text{Model} \rightarrow \text{Simulate} \rightarrow \text{Evaluate Outcomes} \rightarrow \text{Estimate}$$

We will begin with a simple coin flip experiment, where the true probability is known. This lets us see why simulation works.

We will then apply the same idea to stock returns and use simulated returns to estimate:

- the probability of a large daily loss,
- the distribution of portfolio profit and loss,
- and a simple one-day Value at Risk measure.

---

2 Monte Carlo Intuition: Coin Flips

Suppose we repeatedly flip a fair coin.

Let

$$X_i = \begin{cases} 1, & \text{if flip } i \text{ is heads,} \\ 0, & \text{if flip } i \text{ is tails.} \end{cases}$$

Because the coin is fair,

$$P(X_i = 1) = 0.5.$$

After N flips, we can estimate the probability of heads using

$$\hat{p}_N = \frac{1}{N} \sum_{i=1}^{N} X_i.$$

As the number of flips becomes large, we expect the estimated probability to approach the true probability:

$$\hat{p}_N \to 0.5.$$

This is the intuition behind Monte Carlo simulation: if we generate enough outcomes from a known model, the simulated outcomes can be used to estimate properties of that model.

2.1 Simulating Coin Flips in R

We can represent heads with `1` and tails with `0`.

The `sample()` function allows us to randomly sample from a set of possible outcomes.

```
flips <- sample(
  x = c(0, 1),
  size = 1000,
  replace = TRUE,
  prob = c(0.5, 0.5)
)

head(flips)
```

```
## [1] 1 1 1 1 1 1
```

The argument `replace = TRUE` is important because each flip is independent. After one flip occurs, both heads and tails remain possible on the next flip.

2.2 Tracking the Estimated Probability

The cumulative number of heads after each flip is

```
cumulative_heads <- cumsum(flips)
```

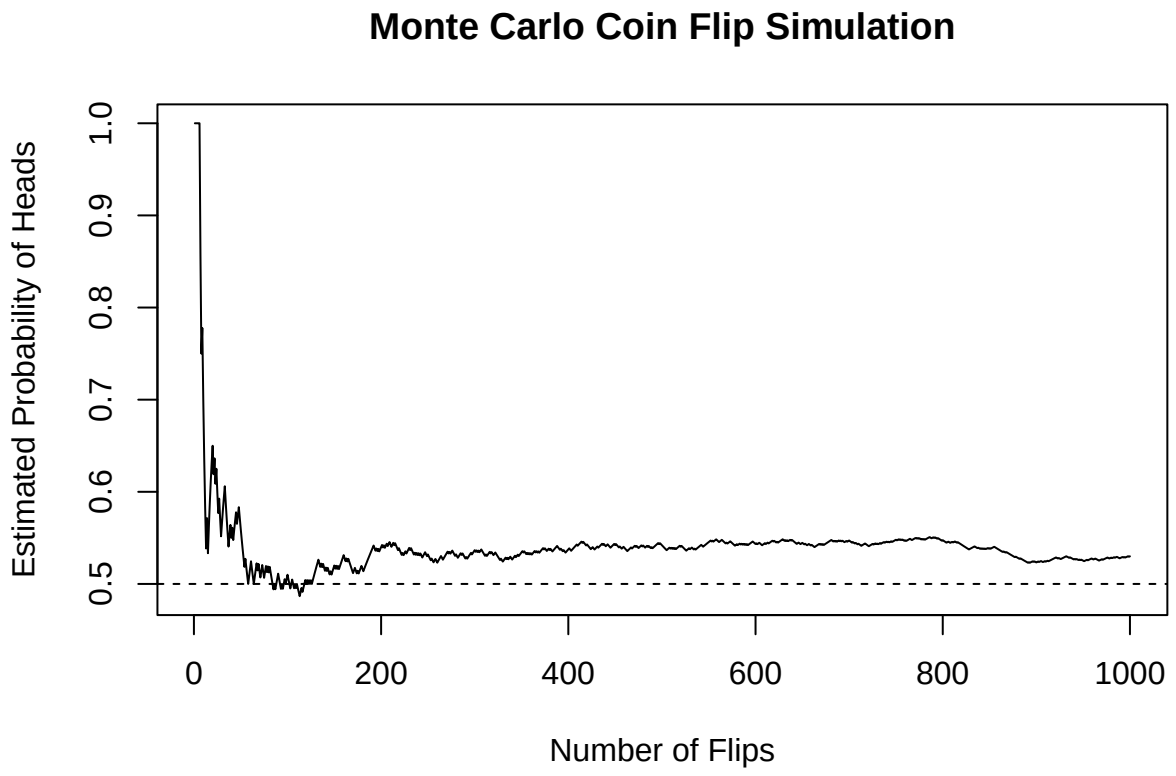
The estimated probability of heads after each flip is therefore

```
running_probability <- cumulative_heads / (1:1000)
```

We can plot this estimate over time.

```
plot(
  1:1000,
  running_probability,
  type = "l",
  xlab = "Number of Flips",
  ylab = "Estimated Probability of Heads",
  main = "Monte Carlo Coin Flip Simulation"
)

abline(h = 0.5, lty = 2)
```



2.2.1 Interpretation

At the beginning, the estimated probability may move substantially because we have very little data.

As the number of simulated flips increases, the estimate tends to stabilize near 0.5.

The main takeaway is:

$$\text{Probability} \approx \frac{\text{Number of Simulated Outcomes of Interest}}{\text{Total Number of Simulations}}.$$

3 Monte Carlo for Financial Returns

The same idea can be applied to financial markets.

Suppose we model a stock's one-day return as normally distributed:

$$R \sim N(\mu, \sigma^2),$$

where

- μ is the expected daily return,
- σ is the daily volatility.

For this example, assume

$$\mu = 0.0005$$

and

$$\sigma = 0.02.$$

This corresponds to an expected daily return of 0.05% and daily volatility of 2%.

```
mu <- 0.0005
sigma <- 0.02
```

4 Simulating Stock Returns

R provides the function `rnorm()` to generate random observations from a normal distribution.

The general form is

```
rnorm(n, mean, sd)
```

where

- `n` is the number of simulations,
- `mean` is the distribution mean,
- `sd` is the standard deviation.

We will simulate 10,000 possible one-day stock returns.

```r
simulated_returns <- rnorm(
  n = 10000,
  mean = mu,
  sd = sigma
)
```

We can inspect the first few simulated returns.

```r
head(simulated_returns)
```

```
## [1]  0.022697273  0.013747157 -0.002907935 -0.002946414 -0.017782506
## [6] -0.019137071
```

We can also compare the simulated mean and standard deviation to the values used in the model.

```r
mean(simulated_returns)
```

```
## [1] 0.0002456952
```

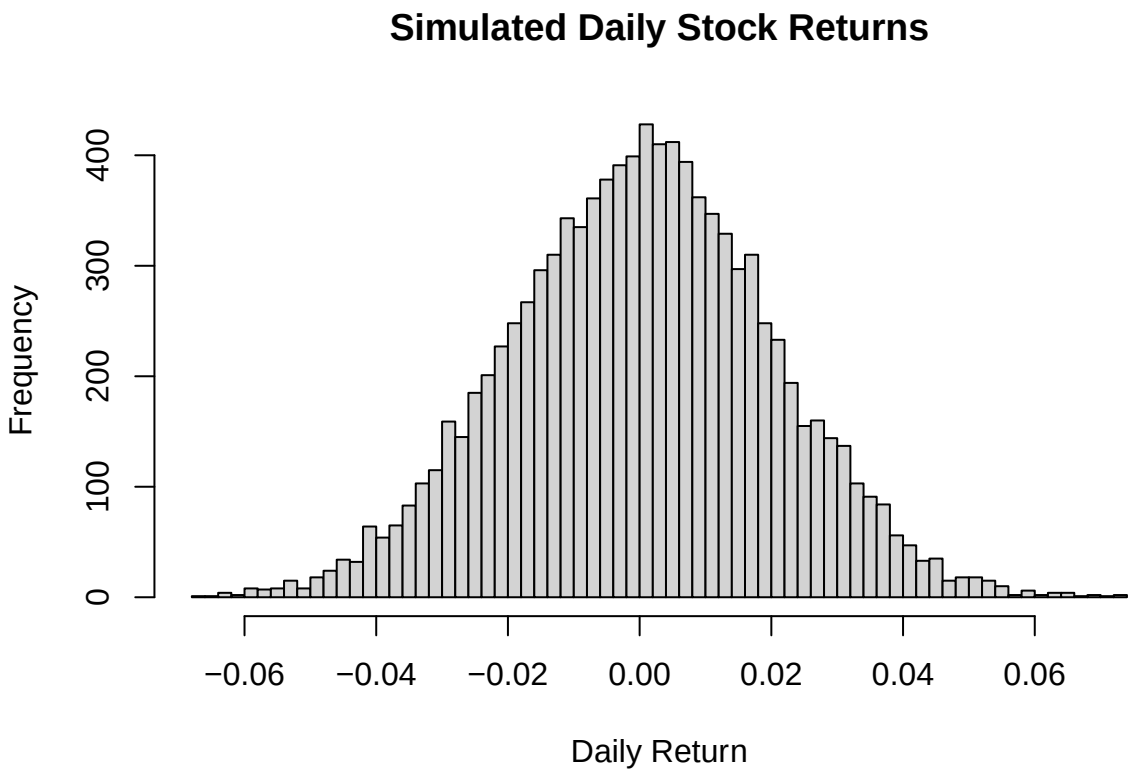
```r
sd(simulated_returns)
```

```
## [1] 0.01986714
```

These will not usually be exactly equal to μ and σ, but with many simulations they should be close.

4.1 Visualizing the Simulated Distribution

```r
hist(
  simulated_returns,
  breaks = 50,
  main = "Simulated Daily Stock Returns",
  xlab = "Daily Return"
)
```



The histogram represents the range of possible one-day returns generated by our model.

5 Estimating a Probability with Monte Carlo

Suppose we want to answer the following question:

> What is the probability that the stock loses more than 3% tomorrow?

Mathematically, we want

$$P(R < -0.03).$$

5.1 Monte Carlo Estimate

We can ask which simulated returns are below -3%.

```
loss_event <- simulated_returns < -0.03

head(loss_event)
```

```
## [1] FALSE FALSE FALSE FALSE FALSE FALSE
```

Each comparison returns either `TRUE` or `FALSE`.

In R,

$$\text{TRUE} = 1, \qquad \text{FALSE} = 0.$$

Therefore, taking the mean gives us the proportion of simulations in which the return was below -3%.

```
prob_loss_mc <- mean(loss_event)

prob_loss_mc
```

```
## [1] 0.0646
```

This is our Monte Carlo estimate:

$$P(R < -0.03) \approx \frac{\text{Number of Simulated Returns Below } -3\%}{\text{Total Number of Simulations}}.$$

Notice that this is exactly the same logic as our coin flip experiment.

---

6 Comparing Monte Carlo to the Analytical Answer

Because we assumed returns are normally distributed, we can also calculate this probability directly.

For a normal random variable, `pnorm()` gives the cumulative probability

$$P(R \leq x).$$

```
prob_loss_exact <- pnorm(
  q = -0.03,
  mean = mu,
  sd = sigma
)

prob_loss_exact
```

```
## [1] 0.06362955
```

Now compare the two estimates.

```
prob_loss_mc
```

```
## [1] 0.0646
```

```
prob_loss_exact
```

```
## [1] 0.06362955
```

They should be reasonably close.

This is useful because the analytical answer is known here, so we can verify that Monte Carlo simulation behaves as expected.

In many more complicated financial problems, however, a simple analytical solution may not be available. Simulation can then become the primary method.

---

7 Simulating Portfolio Profit and Loss

Suppose we invest $100,000 in the stock.

```
portfolio_value <- 100000
```

For a one-day return R, the approximate dollar profit or loss is

$$\text{PnL} = V_0 R.$$

We can apply this calculation to every simulated return.

```
simulated_pnl <- portfolio_value * simulated_returns
```

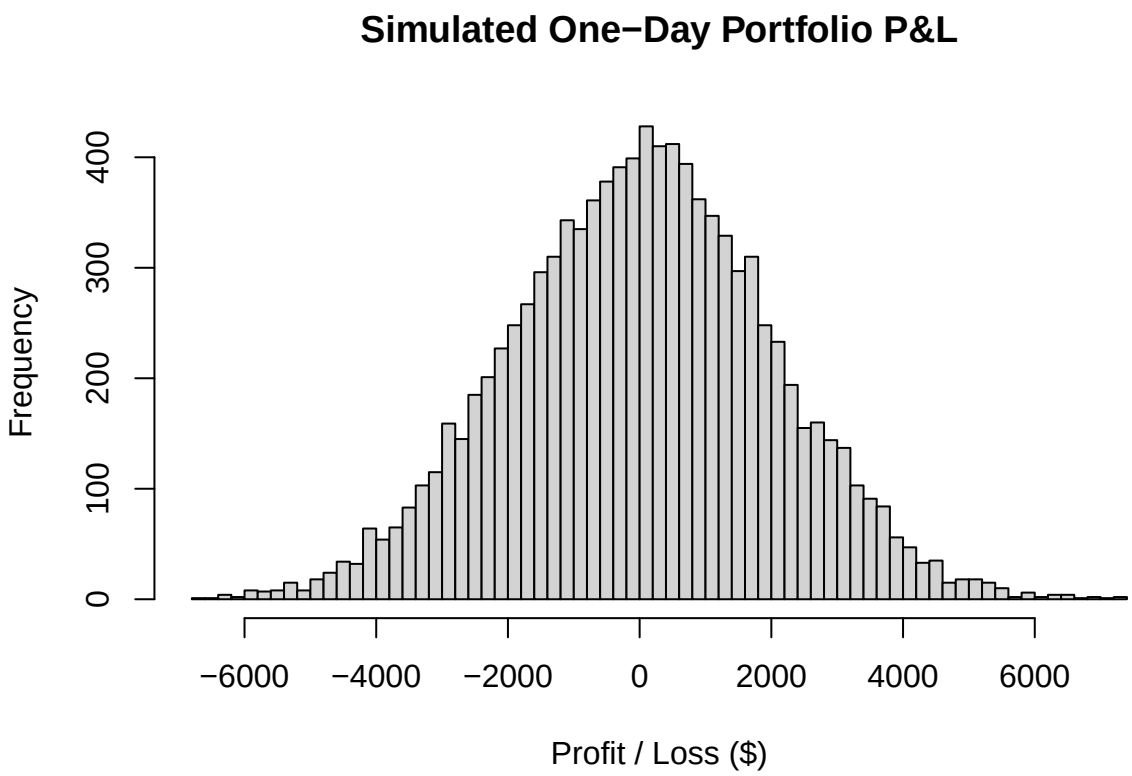
Inspect a few possible outcomes.

```
head(simulated_pnl)
```

```
## [1]  2269.7273  1374.7157  -290.7935  -294.6414 -1778.2506 -1913.7071
```

And visualize the simulated P&L distribution.

```
hist(
  simulated_pnl,
  breaks = 50,
  main = "Simulated One-Day Portfolio P&L",
  xlab = "Profit / Loss ($)"
)
```



Each value represents one possible one-day outcome for the portfolio under our assumed return model.

8 A Simple Value at Risk Calculation

Suppose we want to identify a loss threshold that is exceeded only about 5% of the time.

The 5th percentile of the P&L distribution is

```
quantile(simulated_pnl, 0.05)
```

```
##        5%
## -3259.933
```

If this value is negative, it represents a loss.

We can define a simple 95% one-day Value at Risk as

```
VaR_95_mc <- -quantile(simulated_pnl, 0.05)

VaR_95_mc
```

```
##       5%
## 3259.933
```

8.0.1 Interpretation

A 95% one-day VaR of, for example, approximately $3,200 would mean:

> Under this model, about 5% of simulated one-day outcomes produce a loss greater than approximately $3,200.

It does **not** mean that $3,200 is the worst possible loss.

It is a percentile of the simulated loss distribution.

9 Analytical VaR with `qnorm()`

For the normal distribution, `qnorm()` gives the value associated with a specified cumulative probability.

In notation,

$$x = F^{-1}(p).$$

For the 5th percentile of the return distribution:

```
return_5pct <- qnorm(
  p = 0.05,
  mean = mu,
  sd = sigma
)

return_5pct
```

```
## [1] -0.03239707
```

Convert this return into a dollar VaR.

```
VaR_95_normal <- -portfolio_value * return_5pct

VaR_95_normal
```

```
## [1] 3239.707
```

Compare the simulated and analytical values.

```
VaR_95_mc
```

```
##       5%
## 3259.933
```

```
VaR_95_normal
```

```
## [1] 3239.707
```

Again, the two should be reasonably close.

10 Summary

The important idea is the Monte Carlo process:

$$\boxed{\text{Model} \rightarrow \text{Simulate} \rightarrow \text{Evaluate Outcomes} \rightarrow \text{Estimate}}$$

For the coin flip:

$$\text{Fair Coin} \rightarrow \text{Simulated Flips} \rightarrow \text{Heads or Tails} \rightarrow \text{Estimated Probability}.$$

For the portfolio:

$$\text{Return Distribution} \rightarrow \text{Simulated Returns} \rightarrow \text{Portfolio P\&L} \rightarrow \text{Probability or VaR}.$$

The R functions introduced in this walkthrough are:

- `sample()` — simulate outcomes from a set of possible values,
- `cumsum()` — calculate cumulative totals,
- `set.seed()` — make random simulations reproducible,
- `rnorm()` — generate normally distributed random variables,
- `pnorm()` — calculate cumulative normal probabilities,
- `qnorm()` — calculate normal-distribution quantiles,
- `quantile()` — calculate sample quantiles from simulated data.

The broader lesson is that R can be used not only to analyze historical data, but also to generate possible future scenarios and quantify uncertainty.